

Ensemble Learning Techniques Project

Student Assignment Submission

1. Introduction to Ensemble Learning

Ensemble Learning is a Machine Learning technique where multiple models are combined together to improve prediction accuracy and performance.

Instead of using a single model, Ensemble Learning combines multiple models and produces better results.

Advantages of Ensemble Learning

- Higher Accuracy
 - Reduces Overfitting
 - Better Performance
 - More Reliable Predictions
-

2. Types of Ensemble Learning Techniques

2.1 Bagging

Bagging stands for Bootstrap Aggregating.

It trains multiple models in parallel using random subsets of data.

Example:

- Random Forest

Advantages:

- Reduces variance
 - Improves stability
-

2.2 Boosting

Boosting trains models sequentially.

Each new model corrects the mistakes of the previous model.

Examples:

- AdaBoost
- Gradient Boosting
- XGBoost

Advantages:

- High Accuracy
 - Better performance on difficult datasets
-

2.3 Stacking

Stacking combines predictions from different algorithms and uses another model to make final predictions.

Example:

- Logistic Regression + Decision Tree + SVM
-

3. Random Forest Algorithm

Random Forest is an Ensemble Learning algorithm based on Bagging.

It combines multiple Decision Trees and gives the final prediction based on majority voting.

Advantages of Random Forest

- High Accuracy
- Handles large datasets
- Reduces overfitting
- Works well for classification problems

Disadvantages

- Slower for large datasets
 - Complex compared to single Decision Tree
-

4. Dataset Used

Dataset Name: Titanic Dataset

Objective: Predict whether a passenger survived or not.

Target Column:

- Survived

5. Python Program – Ensemble Learning Project

File Name: ensemble_learning.py

```
# =====  
# Ensemble Learning Project using Random Forest  
# =====  
  
# Import Libraries  
import pandas as pd  
import numpy as np  
import joblib  
  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import LabelEncoder  
  
from sklearn.linear_model import LogisticRegression  
from sklearn.tree import DecisionTreeClassifier  
from sklearn.neighbors import KNeighborsClassifier  
from sklearn.ensemble import RandomForestClassifier  
  
from sklearn.metrics import accuracy_score  
from sklearn.metrics import classification_report  
from sklearn.metrics import confusion_matrix  
  
# =====  
# Load Dataset  
# =====  
  
# Replace with your dataset file name  
  
data = pd.read_csv('titanic.csv')  
  
print("First 5 Rows")  
print(data.head())  
  
# =====  
# Data Preprocessing  
# =====  
  
# Remove unnecessary columns  
  
data = data.drop(['PassengerId', 'Name', 'Ticket', 'Cabin'], axis=1)
```

```

# Fill missing values

data['Age'].fillna(data['Age'].mean(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)

# Convert categorical columns into numerical values

label_encoder = LabelEncoder()

data['Sex'] = label_encoder.fit_transform(data['Sex'])
data['Embarked'] = label_encoder.fit_transform(data['Embarked'])

# =====
# Features and Target
# =====

X = data.drop('Survived', axis=1)
y = data['Survived']

# =====
# Train Test Split
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# =====
# Logistic Regression
# =====

log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train, y_train)

log_pred = log_model.predict(X_test)

log_accuracy = accuracy_score(y_test, log_pred)

print("\nLogistic Regression Accuracy:", log_accuracy)

# =====
# Decision Tree
# =====

dt_model = DecisionTreeClassifier()
dt_model.fit(X_train, y_train)

# Predictions

dt_pred = dt_model.predict(X_test)

```

```

dt_accuracy = accuracy_score(y_test, dt_pred)

print("Decision Tree Accuracy:", dt_accuracy)

# =====
# KNN Algorithm
# =====

knn_model = KNeighborsClassifier()
knn_model.fit(X_train, y_train)

knn_pred = knn_model.predict(X_test)

knn_accuracy = accuracy_score(y_test, knn_pred)

print("KNN Accuracy:", knn_accuracy)

# =====
# Random Forest Algorithm
# =====

rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

rf_pred = rf_model.predict(X_test)

rf_accuracy = accuracy_score(y_test, rf_pred)

print("Random Forest Accuracy:", rf_accuracy)

# =====
# Model Evaluation
# =====

print("\nClassification Report")
print(classification_report(y_test, rf_pred))

print("\nConfusion Matrix")
print(confusion_matrix(y_test, rf_pred))

# =====
# Accuracy Comparison
# =====

print("\nModel Comparison")
print("Logistic Regression:", log_accuracy)
print("Decision Tree:", dt_accuracy)
print("KNN:", knn_accuracy)
print("Random Forest:", rf_accuracy)

# =====

```

```
# Save Random Forest Model
# =====

joblib.dump(rf_model, 'random_forest_model.pkl')

print("\nModel Saved Successfully")
```

6. Streamlit Front-End Application

File Name: app.py

```
# =====
# Streamlit Front-End for Random Forest
# =====

import streamlit as st
import joblib
import numpy as np

# Load Saved Model
model = joblib.load('random_forest_model.pkl')

# Title
st.title("Titanic Survival Prediction")

st.write("Enter Passenger Details")

# Input Fields
Pclass = st.selectbox("Passenger Class", [1, 2, 3])
Sex = st.selectbox("Sex (0 = Female, 1 = Male)", [0, 1])
Age = st.number_input("Age", min_value=1, max_value=100)
SibSp = st.number_input("Siblings / Spouse", min_value=0, max_value=10)
Parch = st.number_input("Parents / Children", min_value=0, max_value=10)
Fare = st.number_input("Fare", min_value=0.0)
Embarked = st.selectbox("Embarked", [0, 1, 2])

# Predict Button
if st.button("Predict"):

    input_data = np.array([
        Pclass, Sex, Age, SibSp, Parch, Fare, Embarked
    ])

    prediction = model.predict(input_data)

    if prediction[0] == 1:
        st.success("Passenger Survived")
```

```
else:  
    st.error("Passenger Did Not Survive")
```

7. How to Run the Project

Step 1: Install Required Libraries

```
pip install pandas numpy scikit-learn streamlit joblib
```

Step 2: Run Python File

```
python ensemble_learning.py
```

Step 3: Run Streamlit App

```
streamlit run app.py
```

8. Output

The model predicts whether a passenger survived or not.

The Streamlit front-end provides:

- User Input Fields
 - Predict Button
 - Prediction Result
-

9. Conclusion

In this project:

- Ensemble Learning concepts were studied.
- Different Ensemble Techniques were learned.
- Random Forest Algorithm was implemented.
- Titanic Dataset was used.
- Model performance was evaluated.

- Random Forest was compared with other algorithms.
- Streamlit front-end was created.
- Trained model was saved and loaded successfully.

Random Forest produced better accuracy compared to other algorithms.